factory, as we write to the ephemeral object that's returned by a call to *qDebug()*. Contrast this with *qFatal()*, in which we passed the message as an argument. This is because *qFatal()* never returns - it crashes the program immediately.

In the next few lines, let's write a random fortune, and then call *disconnectFromHost()*. Yes, that's *disconnectFromHost()*, not *disconnect()*, because *disconnect()* on any QObject-derived class (which is pretty much all the classes in Qt) disconnects all signals and slots connected to the object.

Finally, we connect the *disconnected()* signal (which is emitted when the socket has finally disconnected) to the *deleteLater()* slot on the same socket. So now, when the socket disconnects, it'll be queued for deletion. This is the standard way of tearing down a socket.

It's important that you don't *write()* and then immediately *close()* and *deleteLater()*. This is because *write()* and *disconnectFromHost()* are asynchronous functions, which only perform the writing and disconnecting after the control passes to the main event loop (which happens when our *sendFortune()* function returns), while *close()* closes the socket immediately, so the socket will have shut down before sending any data out.

We'll now have to fill up *main.cpp*, because we'll need to initialise a FortuneServer object and run it somewhere. The code snippet for *main.cpp*, which is very short, is shown below:

```
#include <QCoreApplication>
#include "fortuneserver.h"

int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);
    FortuneServer f;

    return a.exec();
}
```

And that's all the coding there is. We've just created a network server in about 40 lines of actual code.

Building and running

You can build from QtCreator itself, obviously. On the menu bar, click *Build->Build All*. You'll have a progress bar on the bottom-right corner tracking the build, and if there are errors, an *Error-messages* pane will pop up at the bottom of the window and show you the compiler output.

The actual program will be located at *../build-ProjectName-Desktop-Debug*, relative to the project directory where all the source code is. You can just open up a terminal, cd into the directory and type in the name of the executable, and you're ready to go.

In this case, cd into *build-FortuneServer-Desktop-Debug*, and type in the following command...

```
$: ./FortuneServer
```

Figure 6: It works!

...to start the server. Open another terminal tab, and type the following:

```
$: telnet 127.0.0.1 56789

and press enter. The output I got was:
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
You will be surprised by a loud noise.
Connection closed by foreign host.
```

And there it is! 'You will be surprised by a loud noise' was my fortune. And it did happen, too, as just a few moments later, my room mate decided to enter the room with a loud push on the door, full-on Kramer-style (you do watch Seinfeld, don't you?), and I looked up with a start. Freaky!

The *qmake* way of building the program is pretty easy too. Just open a terminal, cd into the FortuneServer project directory, and type in the following:

```
$: qmake
$: make
```

You'll see some output from g++, and it'll be done. If you list the files in the directory, you'll see a *makefile* that was generated by *qmake*, and *.cpp* files whose names start with *moc_*. These files have been generated by the MOC, so if you want to know what goes on behind the scenes with Qt, you can just go ahead and look at those files now. And the compiled executable is sitting in the same directory, so go ahead and run it!

What now?

First of all, if you need access to the complete code, it's available online on my GitHub account, at *https://github.com/BaloneyGeek/FortuneServerExample*. It builds, so just clone and *qmake && make* to build.

In the next (and final) article in the series, we'll try out something that's fun and build a small application that fetches a fortune and displays it on the screen—in a shiny new GUI. END

By: Boudhayan Gupta

Describing himself as a 'retard by choice', the author believes that madness is a cure-all for whatever is wrong or right with society. A social media enthusiast, he can be reached at *@BaloneyGeek* on Twitter.